Kasi Yedumati · Full Stack Training

Lesson 9 — Frontend and CORS

Course: FastAPI for Full Stack · **Session 9 of 10** · Learning path · **Previous:** Lesson 8 · **Next:** Lesson 10

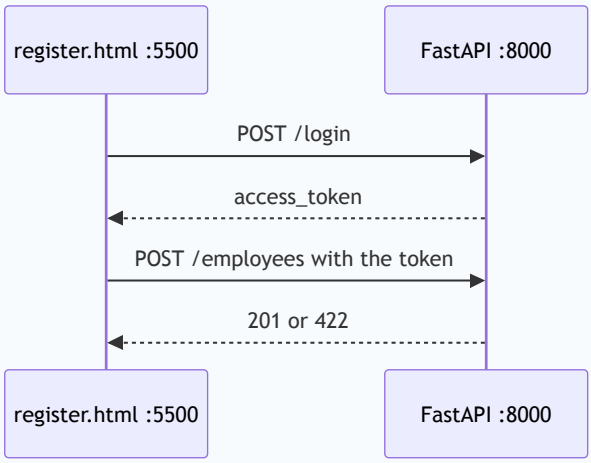
> **Module Outcome:** You can submit the AcmeHR registration form in the browser and show success or the API error on the page.

You will build	`register.html` calling `POST /employees`
New words	CORS, fetch
You need	`fastapi dev main.py` with `/login` and `GET /me`

From the requirements. HR registers an employee from a page, not from `/docs`. The page and the API are two programs. The browser will block the call until the API says this page's address is allowed.

Creating an employee requires HR. The page has two forms. HR types an email and password into the first to log in. The second posts the registration with the token. The page is `examples/lesson09/register.html`.

1. The page and the API are different origins



The page is served from port **5500**. The API listens on port **8000**. The browser treats those as two sites. A page may not call the other site until the API sends a CORS header that names the page's origin.

In `main.py`, after `app = FastAPI(...)`:

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5500", "http://localhost:5500"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

`allow_origins` is the list of pages that may call this API. `allow_headers` includes `Authorization`, which carries the token.

Do not open the HTML file by double-clicking it. The address would start with `file://`. The page's origin is then `null`, which is not in `allow_origins`, so the browser blocks the response from `http://127.0.0.1:8000`. In the folder that contains `register.html`:

```
python3 -m http.server 5500
```

Leave `fastapi dev` running in the other terminal. Open http://127.0.0.1:5500/register.html.

---

2. What the page sends

The login form sends what HR typed. The token goes into a variable. The registration form sends JSON with that token.

```
let token = null;

// Login form: the inputs are named username and password, the names /login expects
const login = await fetch("http://127.0.0.1:8000/login", {
  method: "POST",
  headers: { "Content-Type": "application/x-www-form-urlencoded" },
  body: new URLSearchParams(new FormData(event.target)),
});
token = (await login.json()).access_token;

// Registration form
const response = await fetch("http://127.0.0.1:8000/employees", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer " + token,
  },
  body: JSON.stringify(data),
});
```

Piece	Job
First `fetch`	Login. The body is form fields, because `/login` expects that
`token`	The `access_token` from `POST /login`, kept in a variable. Reloading the page logs you out
Second `fetch`	The registration JSON
`response.ok`	True for 200 and 201. False for 401, 403, and 422

Never write a password into page code. Anyone can open **View Source** and read it, and the page would log in as HR for whoever opens it. The person types the password. The page keeps only the token, and only while the tab is open.

Log in as Rahul (`rahul.sharma@acme.example`, `Acme#1045`). The page writes `Logged in.` Then register. On success the page writes `Created Sana Qureshi (E-1048)`. On a second click the API returns **400** because the id exists, and the page writes that status and `detail`.

Clear the name and submit. The name box has no `required` attribute on purpose, so the empty name reaches the API. The page shows **422** and `String should have at least 1 character`.

Log in as Ananya instead and register a new id. The page shows **403** and `HR only`.

Common error: The browser console says the request was blocked by CORS. The origin in the address bar is not in `allow_origins`. `localhost` and `127.0.0.1` are different origins. The list above includes both.

Common error: `Login failed` on the page while `/docs` login works. The API answered **401**: the email or password typed on the page does not match a seeded user. Compare them with the table in Lesson 8.

Common error: The page writes `Log in first.` You reloaded the page, which cleared the token. Log in again.

Common error: The page stays on `Logging in…` or `Sending…`. The request never got an answer. Open the browser console: either CORS blocked it, or `fastapi dev` is stopped.

Try these

1. **(Easy)** Register `E-1048` from the page. GET that id in `/docs`.
2. **(Easy)** Submit again. The page shows **400**.
3. **(Medium)** Remove `Authorization` from the second `fetch`, save, submit a new id. Status **401**.
4. **(Stretch)** Serve the file on port `5501` without changing `allow_origins`. Read the CORS message in the browser console. Add that origin and submit again.

Interview: *Why does the browser block the call?* — The page and the API have different origins. The API must name the page's origin in CORS before the browser will hand the response to the page.

Key takeaways

- Serve the HTML on port 5500. Keep the API on port 8000.
- `CORSMiddleware` lists the origins that may call the API.
- The page logs in, then sends `Authorization: Bearer` with the JSON.
- `response.ok` chooses the success sentence or the error text.

Lab — Ticket ACME-209

The registration page must create an employee and show an error when the API rejects the body.

```
PYTHON/17_FastAPI/labs/L09_frontend/
├── broken/      ← Part A. The API and the page each have bugs.
├── starter/     ← Part B. No CORS yet, and the page has no script.
└── solution/    ← Open only after the checklist passes.
```

Each folder has `main.py` and `register.html`. In that folder, copy `.env.example` to `.env`, run `fastapi dev main.py` in one terminal and `python3 -m http.server 5500` in another.

Part A — Fix the broken page and API (10 min)

`broken/` has **3 bugs**. Keep the browser console open.

1. Open `http://127.0.0.1:5500/register.html` and log in as Rahul. The page stays on `Logging in…`, and the console reports a CORS error. The same page on `http://localhost:5500` works.
2. Register Sana. The page says `Created undefined (undefined)`, and MySQL has no new row. The page claims success whatever the status.
3. After you fix 2, the page shows **422** and `Input should be a valid dictionary or object`. Compare the headers of the second `fetch` with section 2.

Part B — Build it (15 min)

In `starter/`, add `CORSMiddleware` to `main.py`. `register.html` has both forms and a `token` variable. Write the two submit handlers from the TODOs.

Part C — Prove it works (5 min)

- [ ] `http://127.0.0.1:5500/register.html` loads. `file://` is not the address.
- [ ] Logging in as Rahul shows `Logged in.` A wrong password shows `Login failed`.
- [ ] A valid new id shows `Created ...` and the row is in MySQL.
- [ ] An empty name shows **422** on the page.
- [ ] Logged in as Ananya, a registration shows **403**.
- [ ] No password appears anywhere in `register.html`.
- [ ] The browser console has no CORS error.

Homework

1. Push `register.html` and the CORS change to your `acmehr-api` repo.
2. **Transfer drill (15 minutes).** Add a second page `whoami.html` that logs in as Ananya and writes the JSON from `GET /me` on the page.

Next: Mini-project. Demonstrate this API from the table to the page.